RISC-V 与 PE 工程
基础命令与排错指南

给刚开始使用 Linux、Design Compiler 和仿真工具的你

这份文档讲的是操作工程的命令：怎么读入电路、设置要求、综合、看报告和仿真。这里的“命令”不是 RISC-V 的 `add`、`lw` 等 CPU 指令。

先记住一条主线：

读入电路 → 设置约束 → 综合 → 看报告 → 导出文件 → 仿真验证

你以前输入的一条 `source xxx.tcl`，通常已经让脚本替你完成了很多条内部命令。接下来要学会判断每一步做了什么、结果放在哪里、通过了什么。

按需要翻阅

页码	内容	主要解决的问题
2—3	终端、路径、脚本	命令在哪输？source 是什么？
4—8	读入、约束、综合、报告、导出	DC 究竟在处理什么？
9—12	仿真与真实工程顺序	怎么按正确顺序使用现有脚本？
13—16	日志、卡住、常见报错、SDF	出问题先看什么？
17	命令速查	忘记命令时快速查找

工程根目录：`/home/zhz/contest/task1/my_design`

本指南结合当前工程的 `project/synth/README.md` 和实际脚本编写。数字和 PASS 记录用于解释已有结果；再次运行时，以本次结果目录和本次报告为准。

目前进度：单 PE、阵列、AXI+PE 已有对应层次的验证记录；全 SoC 已完成读入和约束保存，CPU/路由控制逻辑的整机综合及后续验收仍待完成。

01 | 命令在哪里输入？先认清两个终端状态

1. 普通 Linux 终端：管理文件、启动工具

屏幕常见提示符是 zhz@机器名:目录$。本文称它为 Bash。提示符不用复制，只复制下面的命令。

```bash
pwd
ls
cd /home/zhz/contest/task1/my_design
```

`pwd` 显示“我现在在哪个目录”；`ls` 列出这个目录里的文件；`cd` 切换目录。这三条不会修改电路。

路径就是文件的地址。 `/home/...` 从根目录开始，是绝对路径；`project/synth/read_pe.tcl` 从当前目录开始，是相对路径。相对路径报错时，先用 `pwd` 查位置。

2. 第一次打开工具的顺序

在普通 Bash 中，依次输入：

```bash
source /home/zhz/contest/env.sh
cd /home/zhz/contest/task1/my_design
command -v dc_shell
dc_shell
```

第一条加载工具环境，让当前终端知道 DC、VCS 等工具在哪。第三条检查是否能找到 DC；第四条启动 DC。看到 `dc_shell>`，才进入电路综合工具。

已有 `dc_shell>` 窗口时，直接用那个窗口即可。 不需要为了每个步骤重新启动一个 DC。

3. Design Compiler：处理电路

在 `dc_shell>` 中，可以输入：

```tcl
current_design
report_area
```

第一条查看当前正在处理哪个顶层；第二条查看其面积。尚未读入设计时，面积命令无法得到有效结果。

看到的状态	该输入什么	典型错误
Bash 的 `$` 提示符	Linux 命令、启动工具	在这里直接输 report_timing
`dc_shell>`	DC/Tcl 命令	在这里把普通 .sh 当 Tcl source

`exit` 会退出当前程序：在 DC 中是退出 DC，未保存的内存设计会丢失。它不是“返回上一条命令”。

02 | source、bash、exec：一条命令为何能做很多事？

1. 脚本就是提前写好的操作步骤

例如，在 DC 中输入：

```tcl
source /home/zhz/contest/task1/my_design/project/synth/read_pe.tcl
```

相当于告诉 DC：“请依次执行这个文件里的命令。”里面会设置库、读 Verilog、展开顶层、连接模块、检查 SRAM，再保存结果。

所以 `source` 不是打开文件看一看，而是执行文件里的步骤。

2. 同一个单词，执行环境不同

任务	在哪里输入	正确写法
加载工具环境	Bash	source /home/zhz/contest/env.sh
执行综合脚本	DC	source read_pe.tcl
执行 Shell 脚本	Bash	bash run_pe_sdf.sh
从 DC 调 Shell 脚本	DC	exec bash run_pe_sdf.sh

表中的短文件名要求当前目录正确；实际操作可用完整路径。在 DC 里，`exec` 表示调用外部程序。在 Bash 里，`exec` 会替换当前 Shell，因此普通终端按本文写 `bash ...` 即可。

`.tcl` 通常是 Tcl 脚本，`.sdc` 是约束脚本，`.sh` 通常交给 Bash。扩展名帮助识别，但最终仍要看文件内容和说明。

3. 看懂 Tcl 的几个符号就够用

```tcl
set period 10
create_clock -name pe_clk -period $period [get_ports clk]
```

第一行把数值 10 记在变量 `period` 中；第二行读取 `$period`，先用 `[get_ports clk]` 找到名为 `clk` 的端口，再对它建立时钟。

`-period 10` 是一个选项及它的值；`$period` 读取变量；`[get_ports clk]` 先执行方括号里的命令，再使用结果。

`{clk rst_n}` 把内容作为一组文字传入，常用于对象列表。行末 `\` 表示下一行仍属于同一条命令，后面不要再加空格。

执行入口要分清：有些 `*_batch.tcl` 专门给子进程使用，末尾会 `exit`。不要在保留中的交互 DC 里随意 source；使用本指南列出的入口脚本。

03 | 读入设计：让工具认识电路和库

1. 先分清三样东西

RTL 是你写的电路描述，例如 `pe_1sram_4mul.v`。标准单元库 提供实际可选的门、寄存器及时间/面积数据。SRAM 库 提供存储宏的信息。

可以理解为：RTL 是设计要求，库是工具能够选用的零件目录。工具要同时认识两者，才能把电路落实为具体单元。

当前单 PE 使用 `standard.db` 和 `sram.db`；读入的 RTL 包括 `rfsp16x8.v`、`Exact_8bit.v`、`adder.v` 和 `pe_1sram_4mul.v`。

2. read_pe.tcl 内部在做什么？

下面是脚本片段，供理解，不要单独复制执行；其中 `$sources` 已由原脚本定义。

```tcl
analyze -library PE_WORK -format verilog \
  -define {SYNTHESIS PE_REFERENCE_MACRO} $sources
elaborate pe_1sram_4mul -library PE_WORK
current_design pe_1sram_4mul
link
```

命令	做什么	类比
`read_db`	读入工艺库数据	打开零件目录
`analyze`	解析 RTL、检查语法	看懂各张电路图
`elaborate`	根据顶层、参数展开模块实例	把各张图组成完整结构
`current_design`	查看或选择正在操作的设计	指定当前工作对象
`link`	把实例与对应模块或库单元连接	确认每个零件都有来源

例如，PE 中实例化了 SRAM，`link` 后还要确认它真正绑定到 SRAM 库单元。只剩一个空模块外壳，并不代表存储器已经实现。

3. 怎么判断这一步成功？

单 PE 读入脚本应打印 `DC_READ_PE_PASS`，并核对一个真实绑定的 SRAM。它没有执行 `compile`，所以此时结构已读入，标准单元映射还没完成。

若提示找不到模块或 `unresolved reference`，先查缺少的 RTL、库路径和模块名。不要删除实例或添加空壳来消掉提示。

04 | 约束：告诉工具“这套电路要达到什么要求”

约束是时间和外部环境的要求，例如时钟多快、输入何时到达、输出接多大负载。加约束不会给 RTL 人为插入延时。

下面来自 `pe_io.sdc`，用于解释现有单 PE 的教学预算，不要直接套给别的顶层。

1. 时钟周期：每隔多久采样一次

```tcl
create_clock -name pe_clk -period 10 \
  -waveform {0 5} [get_ports clk]
set_clock_uncertainty 0.1 [get_clocks pe_clk]
set_clock_transition 0.005 [get_clocks pe_clk]
```

当前库时间单位是 ns。周期 10 ns 对应 100 MHz；波形 `{0 5}` 表示 0 ns 上升、5 ns 下降，下一次上升在 10 ns。

这不表示 CPU 每周期一定完成一条指令，也不表示电路中每条路径都恰好耗时 10 ns。`uncertainty` 是为时钟不确定因素留的余量；`transition` 描述边沿转换时间。

2. 输入输出：分配芯片内外的时间

脚本中的 `$pe_data_inputs` 已选出除时钟、复位以外的数据/控制输入：

```tcl
set_input_delay -clock pe_clk -max 1.0 $pe_data_inputs
set_input_delay -clock pe_clk -min 0.0 $pe_data_inputs
set_output_delay -clock pe_clk -max 1.0 [all_outputs]
set_output_delay -clock pe_clk -min 0.0 [all_outputs]
set_input_transition 0.02 $pe_data_inputs
set_load -pin_load 5.0 [all_outputs]
```

输入 `max 1.0 / min 0.0` 表示假设外部发送端在参考时钟沿后 0 ~ 1 ns 把数据送到这个模块。输出 `max 1.0` 表示给外部接收路径预留 1 ns，不能理解为“内部只准用 1 ns”。

`set_load 5.0` 在本工程单位下是每个输出外接 5 fF。数值必须结合 `report_units` 理解，不能跨库机械照抄。

3. 为什么复位单独设置？

当前练习假设 `rst_n` 在下降沿释放，并保留恢复/移除检查。复位不是普通数据，不能为消除违规随意把它设成不检查的路径。

单 PE 执行 `constrain_pe.tcl` 后看到 `DC_PE_CONSTRAINTS_APPLIED`，只表示约束已加上；是否满足，还要重新综合并看报告。

05 | 综合：把 RTL 落实为具体门和寄存器

1. compile 的作用

```tcl
compile
```

这条 DC 命令按照当前约束，把逻辑优化并映射到库里的标准单元。它不是编译 C 程序，也不是运行仿真。

可以理解为：工具原本只知道“这里要做加法、那里要保存数据”，综合后决定“具体用哪些加法逻辑、哪些寄存器、怎样连接”。SRAM 则要保留并绑定对应宏。

映射完成 mapped，表示逻辑已选用具体库单元；`unmapped` 表示还有通用逻辑未落实。读入成功与映射成功是两个不同阶段。

2. 为什么 setup 和 hold 都要看？

Setup，建立时间：采样时钟沿到来前，数据要提前稳定。路径太慢，数据来不及到达，就可能违规。

Hold，保持时间：采样时钟沿到来后，旧数据还要稳定一小会儿。路径太短，新数据来得太早，也可能违规。

所以电路不能只追求“所有地方越快越好”。原工程就曾在 setup 满足时出现 hold 余量约 −0.101376 ns。

现有 `recompile_pe.tcl` 内部执行：

```tcl
set_fix_hold [get_clocks pe_clk]
compile
```

第一条提出保持时间修复要求，第二条实际综合。已有结果通过加入真实延迟单元修复了短路径，没有通过放宽输入预算来掩盖问题。

3. 改了约束，为什么还要重综合？

因为之前的单元选择和连接，是按旧要求做的。只加载新 SDC，报告可能变了，但网表并没有自动重新优化。

本工程推荐使用经过检查的 `recompile_pe.tcl`，它会加完整教学约束、综合、检查映射/SRAM、保存报告和文件。不要在阵列或 SoC 顶层误用单 PE 脚本。

`set_dont_touch` 用于保护指定电路不被重新优化；阵列/AXI 阶段会保护已验证的 PE。遇到它时先理解保护边界，不能随便取消后把所有 PE 重综合。

06 | 报告：怎样判断结果好不好？

在已加载正确设计并应用相应约束的 `dc_shell>` 中查看：

```tcl
report_area
report_timing -delay_type max -max_paths 3
report_timing -delay_type min -max_paths 3
report_constraint -all_violators
```

命令或报告	主要回答什么
`report_area` / area.rpt	选用单元的面积合计有多大？
`delay_type max` / setup.rpt	最慢路径是否来得及？
`delay_type min` / hold.rpt	最短路径是否变得过早？
`report_constraint -all_violators`	当前检查到哪些约束违规？
`check_timing`	时钟、接口预算等是否有缺失？

`-max_paths 3` 让报告展示代表性的最差路径，不是说电路只存在三条路径。面积也只是当前库条件下的单元估算，不等于最终芯片面积。

1. 看时序报告，先抓住四处

`Startpoint` 是起点；`Endpoint` 是终点；`Path Type` 告诉你在看 max 还是 min；`slack` 是余量。

对 setup 而言，可以类比“要求 10 秒内交作业”：8 秒交还有 +2 秒，11 秒交就是 −1 秒。对 hold 则要理解为“数据别换得过早”，不能直接套迟到的例子。

slack	通俗判断	接下来做什么
正数	当前要求下还有余量	继续核对其他路径、接口及违规
0 或非常接近 0	时间刚好或余量很小	留意模型、布线、时钟变化
负数	当前要求下有违例	查具体路径、约束及修复方法

2. 结合真实结果理解

单 PE 完整教学约束后的已有记录：setup 为 +6.12428 ns，hold 为 +0.00240409 ns，后者只有约 2.4 ps。已修复当前违规，但余量很小。

这些是当前工程库、约束和综合估计下的结果。尚无物理时钟树和真实布线寄生，不能写成“芯片所有工况时序全部通过”。

若显示 `This design has no violated constraints.`，还要确认本该检查的时钟、输入输出确实都被约束。漏掉要求，也可能看不到违规。

07 | 导出和保存：每种文件装的是什么？

1. 别只看文件后缀判断阶段

文件	装的内容	主要用途
RTL `.v / .sv`	行为、寄存器和模块的电路描述	综合的输入
映射网表 `.v`	具体库单元和连接关系	门级仿真、后续实现
`.ddc`	DC 保存的设计、层次及相关属性	在 DC 中继续处理
`.sdc`	时钟、接口延迟等约束	重用设计的要求
`.sdf`	门和连线的延迟、时序检查数据	带延迟的门级仿真
`.rpt`	某项分析的报告	查看面积、时序、违规
`.log`	工具执行过程及诊断	找报错和运行记录
`summary.json`	某次运行的结构化汇总	查状态、计数及结果

同样是 `.v`，既可能是 RTL，也可能是已映射网表，还可能混有未映射逻辑。 要看来源、内容和映射检查。

2. 脚本里实际怎样保存？

以下来自现有脚本，变量由脚本设置，供理解，不要单独照抄运行：

```tcl
write -format verilog -hierarchy \
  -output [file join $output_dir pe_mapped.v]
write -format ddc -hierarchy \
  -output [file join $output_dir pe_mapped.ddc]
write_sdc [file join $output_dir pe_constraints.sdc]
write_sdf -version 2.1 -significant_digits 6 $sdf_file
```

`write` 是保存电路，`write_sdc` 是保存要求，`write_sdf` 是保存延迟。导出不会替你重新综合，也不等于已经跑过仿真。

3. 保存 DDC 为什么有用？

它类似工具的检查点。可以用 `read_ddc` 读回，然后选择正确 `current_design` 并连接相应库，继续后面的工作。读回本身不重新综合。

DDC 仍可能依赖库和父结果目录中的设计；当前整机流程尤其要保留相关目录。不要以为复制一个 DDC 到别处，就一定能独立恢复全部工程。

网表和 SDF 要来自同一版设计。 电路重新综合后，旧 SDF 不能随意套到新网表上；实例名和路径一旦不对应，反标就可能失效。

08 | 仿真：电路真的算对了吗？

综合回答“怎样用单元实现”；仿真给电路输入，再检查输出。两者作用不同。

1. 三种检查，分别证明什么？

类型	仿真对象	主要检查
RTL 仿真，又叫前仿	原始 Verilog/SystemVerilog	设计描述的功能是否正确
门级功能仿真	综合后的网表	映射后是否仍保持预期功能
SDF 门级仿真	网表加延迟	带延迟时功能、采样及时序检查是否符合要求

仿真由 testbench，测试平台 提供时钟、复位、读写和参考结果。单 PE/AXI 测试平台可以直接发送操作；它们通过，不等于 CPU 已经运行 C 程序。

2. 当前工程的 RTL 入口

需要运行你的工程前仿时，在 Bash 使用：

```bash
bash /home/zhz/contest/task1/run_rtl.sh \
  --source /home/zhz/contest/task1/my_design
```

这里两行是一条命令，行末 `\` 表示续行。必须指明 `--source`：此入口默认指向 `task1/base`，省略后可能验证的是原始基线，不能据此判断你的修改通过。

3. SDF 要确实加载进仿真器

“SDF 文件存在”和“反标成功”是两件事。反标就是把文件中的延迟加载到对应单元和连线。需要看反标日志、实际加载计数、诊断和延迟是否生效。

SDF 使用 `MAXIMUM` 表示选择三元延迟中的最大值，不代表覆盖所有电压、温度和工艺角。当前使用的是工程库与综合估计延迟。

功能检查中可能使用 `+nospecify`。SDF 时序检查需要保留对应时序模型；不能通过加 `+nospecify` 或关闭 timing checks 来把时序失败“变成通过”。

4. PASS 的范围要看清

现有回归会检查参考结果，再比较 RTL 与门级记录。仅仅“两边输出一样”还不够，因为可能一起算错。

单 PE、16×16 阵列、AXI+PE、全 SoC 是四个不同范围。前一个范围的 PASS，不能直接代替后一个范围的验收。

09 | 实际顺序 A：单 PE，从读入到带延迟验证

本页是在需要重新跑单 PE 时查顺序，不用现在把所有命令再执行一次。前提：库已准备好，在 `dc_shell>`，按步执行并检查结果。

1. 先读入，再按完整教学约束综合

```tcl
source /home/zhz/contest/task1/my_design/project/synth/read_pe.tcl
```

检查 `DC_READ_PE_PASS`，确认顶层为 `pe_1sram_4mul`、SRAM 数量为 1。

```tcl
source /home/zhz/contest/task1/my_design/project/synth/recompile_pe.tcl
```

此脚本内部会调用约束脚本，不必再单独加一次约束。检查 `DC_PE_IO_SYNTHESIS_DONE`，然后看 `PE_IO_SETUP_SLACK_NS`、`PE_IO_HOLD_SLACK_NS` 和 `violations.rpt`。

2. 先门级功能，再带 SDF 检查

```tcl
exec bash /home/zhz/contest/task1/my_design/project/synth/run_pe_functional.sh
```

当前这步的成功标志是 `PE_GATE_FUNCTIONAL_PASS`。

```tcl
exec bash /home/zhz/contest/task1/my_design/project/synth/run_pe_sdf.sh
```

当前这步的成功标志是 `PE_GATE_SDF_PASS`。记录各次 `RESULT=`，功能检查和 SDF 检查的结果目录不同。

3. 之前为什么还学过另外三条 source？

以下是教学时把过程拆开的入口：

```tcl
source /home/zhz/contest/task1/my_design/project/synth/pe_clock.sdc
source /home/zhz/contest/task1/my_design/project/synth/compile_pe.tcl
source /home/zhz/contest/task1/my_design/project/synth/constrain_pe.tcl
```

它们分别是“只加时钟 → 初次综合 → 补接口要求”，帮助观察约束变化。`compile_pe.tcl` 要求当前是单 PE 且已有 `pe_clk`；完整重跑可按本页第 1—2 步，不必重复所有教学中间步骤。

文件位置：初次教学结果在 `project/build/dc_pe`；完整接口约束结果在 `project/build/dc_pe_io`；独立仿真在 `task1/work/pe_functional.*` 和 `pe_sdf.*`。

10 | 实际顺序 B：阵列与 AXI 控制层

为缩短路径，本页先在 DC 输入一次：

```tcl
cd /home/zhz/contest/task1/my_design/project/synth
```

以下短文件名均相对于这个目录。两组步骤各自按顺序，一步检查通过后再往下做；不是要求每次重新跑全部流程。

1. 单 PE 组成 16×16 阵列

前提：已有单 PE 完整约束综合结果。默认阵列有 256 个 PE、256 个 SRAM，复用已映射的 PE。

在 DC 逐条输入	这一步应检查的标记
`source read_pe_array.tcl`	DC_READ_PE_ARRAY_PASS
`source constrain_pe_array.tcl`	DC_PE_ARRAY_CONSTRAINTS_APPLIED
`source export_pe_array.tcl`	DC_PE_ARRAY_EXPORTED
`source run_pe_array_sdf.tcl`	PE_ARRAY_GATE_SDF_PASS

这一阶段不要求把 256 个 PE 展平后重新 compile。已映射单 PE 会被复用；读入、连接、阵列约束、导出和阵列行为检查各有独立作用。

2. 给阵列加上 AXI 读写控制

前提：阵列结果已保存。对应顶层为 `sram_multiplier_system_ID_WIDTH3`，不是 `soc_top`。

在 DC 逐条输入	这一步应检查的标记
`source read_pe_axi.tcl`	DC_READ_PE_AXI_PASS
`source constrain_pe_axi.tcl`	DC_PE_AXI_CONSTRAINTS_APPLIED
`source compile_pe_axi.tcl`	DC_PE_AXI_SYNTHESIS_DONE
`source run_pe_axi_functional.tcl`	PE_AXI_GATE_FUNCTIONAL_PASS
`source export_pe_axi.tcl`	DC_PE_AXI_EXPORTED
`source run_pe_axi_sdf.tcl`	PE_AXI_GATE_SDF_PASS

AXI 控制逻辑需要综合，同时保护已验证阵列。功能检查关注读写、计算和复位；SDF 检查再加入延迟。

`APPLIED` 表示要求已设置，`EXPORTED` 表示文件已写出，`DONE` 表示该脚本结束。它们都不能单独替代时序报告或仿真验收。

运行 `run_*` 外部检查后，当前交互 DC 的设计可能保持原样。这些入口按保存文件启动独立任务，不一定把结果装进你当前会话。

11 | 实际顺序 C：全 SoC 现在走到哪里？

1. 模块范围逐级变大

单 PE：一个计算单元。

PE 阵列：把 256 个 PE 连起来。

AXI+PE：阵列外加读写控制，可以接收总线访问。

全 SoC：再接上 CPU 和地址路由，由 CPU 执行程序并访问 PE。

这就像先验证一台机器，再验证车间，最后验证整个工厂的调度和运转。小模块通过是整机的基础，但整机仍有新的连接和控制需要验证。

2. 当前可用的整机入口

在 DC 中使用：

```tcl
source /home/zhz/contest/task1/my_design/project/synth/run_soc_read.tcl
```

成功标志为 `DC_READ_SOC_PASS`。它读入 CPU/路由和已有 PE 电路，检查 256 个 PE、256 个绑定 SRAM；已有一次运行约 10 分钟，时间会随机器状态变化。

```tcl
source /home/zhz/contest/task1/my_design/project/synth/run_soc_constraints.tcl
```

成功标志为 `DC_SOC_CONSTRAINTS_APPLIED`，记录周期 10 ns、152 位数据输入、224 位输出；已有运行约 2 分钟。时钟和复位另行处理。

两条入口会显示进度并保存独立结果，保留交互 DC。外部整机任务成功后，不要假设当前 `current_design` 已切成 soc_top。

3. 哪些已经做过，哪些还没完成？

基础六类操作——读入、约束、综合、报告、导出、仿真——你都已在相关模块上接触过。

但当前全 SoC 的 CPU/路由仍有 181160 个未映射通用叶对象，尚待综合。已有整机 `.v` 不能作为最终标准单元网表使用，下一阶段要从对应 DDC 继续。

后续仍需：整机综合与面积/时序检查；配套网表/SDC/SDF 导出；CPU 运行 C 程序访问 PE 的门级功能验证；整机带延迟验证和结果整理。

目前没有已验证的整机综合入口可照抄。本页不虚构 `compile_soc.tcl`。旧 `read_design.tcl` 展平路径曾内存不足，当前采用上面的新入口。

12 | 日志怎么找？先找到本次 RESULT

1. 一个运行目录，就是一次实验档案

例如终端出现：

```text
RESULT=/home/zhz/contest/task1/work/pe_sdf.ojvwxkha
```

这是历史超时案例的路径，不是新的成功结果。你要记录自己这次屏幕上的 `RESULT=`；后缀每次不同，不能只打开旧目录看见 PASS 就判断本次通过。

在另一个 Bash 终端中，用本次实际路径执行 `cd`。然后：

```bash
pwd
ls -lh
cat summary.json
```

`ls -lh` 看文件大小及时间；`cat` 适合短文件。部分失败可能来不及生成汇总，就直接看已有日志。

2. 大日志不要全部打印，先看末尾和关键词

进入本次结果目录后，常用：

```bash
tail -n 60 gate/run.log
tail -n 60 gate/compile.log
rg -n 'Error|FAILED|timeout|VIOLAT|Warning' gate
```

`tail -n 60` 看最后 60 行；`rg -n` 搜索关键词并显示行号，竖线表示“任意一个关键词”。它默认区分大小写；想忽略大小写可用 `rg -ni`。机器没有 `rg` 时可用 `grep -nEi`。

想查什么	常见位置
最终状态、检查计数	summary.json
RTL 编译或运行出错	rtl/compile.log、rtl/run.log
门级编译或运行出错	gate/compile.log、gate/run.log
SDF 加载与覆盖	gate/sdf_annotation.log、gate/sdfAnnotateInfo
独立 DC 导出问题	export/dc_export.log（具体以该次目录为准）
故意违规实验	timing_probe/run.log

不是每个脚本都有全部文件。看不到某个文件，先 `ls` 确认，不要直接判断它被删了。

报告求助时带上四样东西：本次完整命令、Bash/DC 环境、本次 RESULT 路径、报错附近和日志末尾的内容。只发“卡住了”无法区分计算、等待和失败。

13 | 为什么一直不动？超时和内存不足怎么分？

1. 先看证据，别连续重启

综合和大 SDF 仿真可能一段时间没有新输出。现有流式入口会显示进度；整机入口约每 15 秒报告一次。无输出本身不能证明卡死。

可在另一个 Bash 终端查看：

```bash
ps -eo pid,ppid,stat,%cpu,%mem,rss,etime,args --sort=-rss | head -n 12
free -h
```

`PID` 是进程号；`%CPU` 是 CPU 使用率；`RSS` 是驻留内存，通常以 KiB 显示；`ETIME` 是运行时长。`free -h` 看系统可用内存。结合进度、日志大小/时间及资源变化判断，不能只凭某一瞬间 CPU 为 0 下结论。

2. 真实案例：240 秒超时

你遇到过：

```text
PE_GATE_SDF_FAILED: ... timed out after 240 seconds
```

那次导出日志已出现 `PE_SDF_EXPORTED`，后面停在子进程的 `dc_shell>`。实际是子 DC 没正确退出，在等输入，外层脚本等到 240 秒才报错。

现有入口已修复：批处理脚本明确成功/失败退出，子进程不再继承交互输入。因此不是简单把 timeout 调大就能解决的问题。

再次碰到超时，先查本次 `RESULT` 的阶段日志：是导出没结束、仿真没结束，还是工具停在提示符。不同原因处理不同，不要反复启动更多重任务。

3. 真实案例：Killed / 内存耗尽

旧整机展平文件约 226 MiB、211 万行；导入时 DC 内存曾达约 13.7 GiB，接近虚拟机总量，最终被系统终止。文件能放进磁盘，不代表处理它只需同等内存。

当前整机入口复用已映射 PE，并分阶段处理 CPU/路由，避免重走旧路径。内存紧张时先减少同时运行的重任务；不要靠删库、删父结果目录或反复重跑旧入口解决。

4. 需要中断时

对自己正在等待的前台命令，可按一次 Ctrl+C 请求中断，再确认是否回到正确提示符。它不等于回滚；被中断的输出不能作为完整结果。不要用 `killall dc_shell` 结束全部 DC，其中可能有仍需保留的会话。

14 | 常见报错：看到什么，先做什么？

屏幕现象	常见含义	第一检查动作
`command not found`	工具环境没加载，或命令输错地方	确认 Bash/DC；Bash 运行 command -v dc_shell
`No such file` / cannot open	文件名或路径不对	pwd、ls 检查；注意 Linux 区分大小写
读取时语法错误	当前工具不支持语法，或文件内容有错	找首条报错的文件与行号，不只看最后一条
`unresolved reference`	实例找不到模块或库实现	检查 RTL 清单、库、SRAM 绑定
`Load the single PE...`	当前顶层不是单 PE	用 current_design 核对；先运行对应读取脚本
提示先创建 `pe_clk`	缺少前置时钟	单 PE 教学初次综合前先加载 pe_clock.sdc
`unmapped` / `VO-12`	尚有逻辑未映射到标准单元	区分读入中间态和最终映射结果
`slack (VIOLATED)`	当前假设下有时序违规	区分 max/min，查起点、终点和约束
仿真出现 `X`	存在未知值，原因需追查	查复位、初始化、驱动冲突、SRAM读写及时序违规
外层 `CMD-013`	DC 调用外部命令失败的包装提示	在 DC 用 error_info 查详情，再看 RESULT 内原始日志

1. 有首条错误时，先处理首条

例如“找不到库”会引出“实例无法连接”，后面再出现面积、时序和导出失败。后面十条可能都由第一条引起，逐个改后续现象容易越改越乱。

2. Error 和 Warning 不是同一个意思

`Error/FAILED` 通常说明步骤未完成；`Warning` 表示工具发现需要留意的情况。警告不等于必然失败，也不能一律忽略。要看具体类型、对象，以及脚本是否已针对它检查。

`check_design` 成功，也可能仍有未使用输出、常量和别名提示。当前整机读入保留了结构提醒，并没有声称零警告。

3. 参数只对支持它的脚本有效

不是所有 `.sh` 都接受 `--timeout`。例如公共 `run_rtl.sh` 有对应参数，而单 PE 的包装脚本没有通用同名选项。要看入口说明，不能把别的工具参数直接加过去。

文件找不到时，先修路径；时序违规时，先查要求和电路；未知值时，先追来源。不要靠取消检查或改期待结果消掉失败。

15 | SDF 和工具警告：这些提示怎么理解？

1. SDF 仿真失败，按三个层次检查

先看配套关系。 是否使用同一份导出对应的网表、SDF 和模型？重新综合后有没有误用旧延迟？

再看反标。 `sdf_annotation.log` 和 `sdfAnnotateInfo` 有没有错误、漏载、无法匹配路径或检查项？不能只看 SDF 文件非空。

最后看运行。 正常测试是否出现建立/保持、恢复/移除违规、notifier 或 X？notifier 是模型用于标记时序违规的信号，可能进一步使输出变成 X。

当前工程有针对旧版工具的兼容副本及逐项审计，保留原件和时序检查。遇到失败应查对应日志，不要手工删负时序检查限值、关 specify 或屏蔽 notifier。

2. 故意违规的探针，和正常测试分开看

`timing_probe/run.log` 是单独对照实验：故意在不合适的时间改变数据，检查仿真器能否报出违例。它的预期诊断用于证明检查器真的工作。

这不表示正常回归允许时序违规。先确认你看的日志属于正常测试还是 probe，再判断是否异常。

3. 工程中实际出现过的警告

提示	通俗含义与处理重点
DDB-24	同名库重复读入相关；核对来源和实际绑定
UIO-92	有多份同名设计候选；确认真正使用的 DDC 身份与层次
TIM-134	高扇出相关估算提醒；报告有模型假设，不能当物理签核
TIM-164	库阈值定义差异相关；保留并理解当前库的限制
VO-4	导出时涉及 assign/tran 等结构；结合具体连接核对
VO-12	导出含未映射组件；不能把这个 .v 当最终门级网表
SDFCOM_IWSBA	当前 AXI 案例涉及纯连线别名；需逐条核对，不能泛化忽略

AXI 的已有 SDF 通过记录保留了 129 条已审查的别名警告，并核对实际加载和代表路径延迟。因此应表述为“这组明确核对过的警告被接受”，不能说“零警告”或“这种警告都没关系”。

16 | 最后一页：基础命令速查

1. 普通 Bash：位置、环境、日志

命令	用法
`pwd`	查看当前目录
cd /完整目录	切换工作位置
`ls -lh`	看文件、大小和修改时间
`source /home/zhz/contest/env.sh`	加载当前工具环境
`dc_shell`	启动综合工具
bash 脚本.sh	执行 Shell 脚本
`cat summary.json`	查看短的结果汇总
tail -n 60 文件.log	看日志最后 60 行
rg -n 'Error|FAILED' 目录	搜索错误，显示行号
`free -h`	查看系统内存情况

2. DC：六类基础操作

阶段	主要命令	操作前要知道什么
读入与连接	analyze、elaborate、link、read_ddc	顶层、RTL、库和来源文件
设置约束	create_clock、set_input_delay、set_output_delay、set_load	时钟、单位、外部接口假设
综合	compile、set_fix_hold	当前设计、约束及保护边界
看报告	report_area、report_timing、report_constraint	报告对应的范围和本轮版本
导出	write、write_sdc、write_sdf	设计是否已映射、文件是否配套
验证	source run_*.tcl 或 exec bash ...sh	测试顶层、输入和 RESULT 目录

常用辅助命令：`current_design` 核对当前顶层；`report_units` 核对单位；source 脚本.tcl 执行现有入口。

使用顺序：先核对当前顶层和阶段，再看本次 RESULT、报告与成功标记。返回提示符或打印 DONE，不能单独证明比赛任务全部完成。

继续查工程细节： `project/synth/README.md`。具体输出与验收，以当前脚本和当前运行记录为准。